Screenshots #1 Create_Stack.

hossamshribati — root@0392935df869: /workspace/lab04 — docker ex...

```
VIEWER_LT_NAME=Lab03-Viewer-Template

UPLOADER_TG_NAME=itmo-444-544-uploader-tg
VIEWER_TG_NAME=itmo-444-544-viewer-tg

UPLOADER_ASG_NAME=itmo-444-544-uploader-asg
VIEWER_ASG_NAME=itmo-444-544-viewer-asg

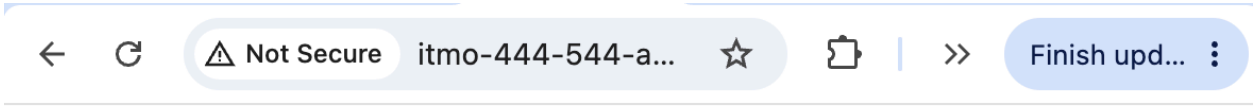
ALB_NAME=itmo-444-544-app-alb
ALB_SG_NAME=itmo-444-544-alb-sg

UPLOADER_LISTENER_PORT=8080
VIEWER_LISTENER_PORT=8081

RECOVERY_SLO_SECONDS=180
root@0392935df869:/workspace/lab04# cd /workspace/lab04
./create_app_stack.sh
Starting Lab 04 stack...
Checking Uploader Auto Scaling Group...
Uploader ASG created.
Checking Viewer Auto Scaling Group...
Viewer ASG created.

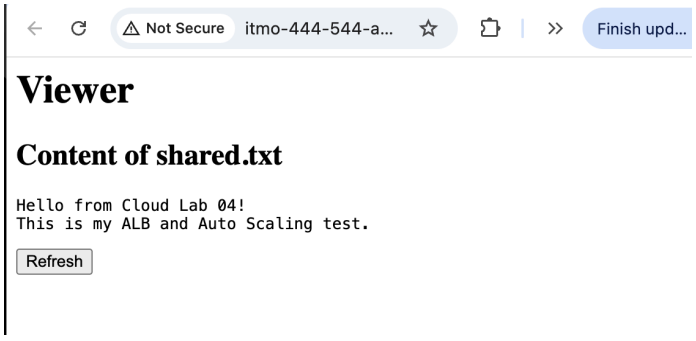
Lab 04 stack creation commands completed!
```

Screenshots # 2 - Uploader Upload successful. And Viewer view



Upload successful.

Upload another



Screenshots # 3 Target_Health

```
hossamshribati — root@0392935df869: /workspace/lab04 — docker ex...

  --target-group-arn arn:aws:elasticloadbalancing:us-east-2:100995727963:targetg
roup/itmo-444-544-uploader-tg/719dea51d85296e3 \
  --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]' \
  --output table

echo "=== VIEWER TARGET HEALTH ==="
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:us-east-2:100995727963:targetg
roup/itmo-444-544-viewer-tg/3d6c0a3377ec89e7 \
  --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]' \
  --output table
=== UPLOADER TARGET HEALTH ===
-------------------------------------
|       DescribeTargetHealth        |
+----------------------+------------+
|  i-070859031c9af06fd |  healthy   |
+----------------------+------------+
=== VIEWER TARGET HEALTH ===
-------------------------------------
|       DescribeTargetHealth        |
+----------------------+------------+
|  i-050a6907fb6cc3d13 |  healthy   |
+----------------------+------------+
root@0392935df869:/workspace/lab04#
```

Screenshot #4: Recovery Time passed

```
128.25s | HTTP 200 | success=True | replacement_healthy=False
133.65s | HTTP 200 | success=True | replacement_healthy=False
138.92s | HTTP 200 | success=True | replacement_healthy=False
144.21s | HTTP 200 | success=True | replacement_healthy=True

CSV saved: recovery_20261018_032531.csv
RESULT: PASS - Recovery time: 144.21s
SLO: 180 seconds
(.venv) root@0392935df869:/workspace/lab04/testing#
```

Screenshot5_Recovery_FAIL test

```
122.78s | HTTP 502 | success=False | replacement_healthy=False
128.04s | HTTP 200 | success=True | replacement_healthy=False
133.3s | HTTP 200 | success=True | replacement_healthy=False
138.53s | HTTP 200 | success=True | replacement_healthy=False
143.8s | HTTP 200 | success=True | replacement_healthy=True

CSV saved: recovery_20261018_033148.csv
RESULT: FAIL - Recovery time: 143.8s
SLO: 60 seconds
(.venv) root@0392935df869:/workspace/lab04/testing#
```

Recovery Time

In my first test, AWS took 144.21 seconds to recover. The test passed because the time limit was 180 seconds. In my second test, AWS took 143.8 seconds to recover. This test failed because the time limit was 60 seconds. Both tests showed that AWS can start a new EC2 instance when the old one stops working.

What Affects Recovery Time

AWS needs time to start a new EC2 instance. The new instance also needs time to install and start the app. After that, the Load Balancer checks if the app is working. All these steps take time. This is why the recovery was not very fast.

Screenshot6_Cleanup

```
echo "===== Verification Complete ====="
echo "===== Verification Complete ====="
===== Lab 04 Cleanup Verification =====
Active EC2 instances:
Auto Scaling Groups:
Load Balancers:
Target Groups:
```